

Lab5: Core Data Structure

COCS-221: Computer Science B



Objective

Practice Python functions and collections by building stacks, selecting dictionary records, and managing a singly linked list.

- Use parameters and return values to write small, testable functions.
- Trace how data changes after each operation.
- Test empty structures, boundary values, and missing records.
- Explain the difference between a Python list and linked nodes.

Overview

Complete Tasks 1 to 3. The two challenges are optional extensions. Start with fixed data and add keyboard input only after your functions work.

Before you begin

Use lists, dictionaries, conditions, loops, and functions. A short refresher before Task 3 explains classes and linked objects.

How to use the starter code

Copy each code block into the named .py file. Keep four spaces for each indentation level. Replace every TODO and NotImplementedError with your implementation. The starters are intentionally incomplete.

Predict first, implement one operation, then test.

None is a special Python value meaning no value is present. In this lab, an empty pop returns None because there is no item to return. None is different from 0, False, and an empty list [].

Task 1 Basic stack operations

File: task1_stack.py

Use a Python list as a stack. Its right-hand end is the top. Before coding, write the expected stack after each operation below.

1. Push 5, then 10, then 15.
2. Pop one number and record the returned value.
3. Check whether the stack is empty.
4. Push 20. Print the stack after every push and pop.

Starter code

```
def push(stack, value):
    # TODO: add value to the top
    raise NotImplementedError

def pop(stack):
    # TODO: return the top number, or None if empty
    raise NotImplementedError

def is_empty(stack):
    # TODO: return True or False
    raise NotImplementedError

stack = []
# TODO: perform the operations and print the results
```

Expected result

After three pushes: [5, 10, 15]. Pop returns 15 and leaves [5, 10]. The empty check returns False. Final stack: [5, 10, 20].

Check and explain

Test an empty stack and a stack with one number. Empty pop must return None. Why does 15 leave first? How does return differ from print?

Task 2 Student marks and a stack

File: task2_students.py

Use five students stored in a dictionary. Names are unique. A mark of 75 or higher qualifies, including exactly 75.

1. `push_qualified`: append qualifying names in dictionary insertion order. Pass in an initially empty stack and call once.
2. `pop_all`: remove every name in LIFO order and return the names in a new list. An empty stack returns [].

Starter code

```
def push_qualified(students, stack):
    # TODO: loop through students.items()
    raise NotImplementedError

def pop_all(stack):
    # TODO: collect and return popped names
    raise NotImplementedError

students = {
    "Vanna": 90, "Tena": 70, "Devi": 75,
    "Mina": 74, "Sok": 100,
}
stack = []
# TODO: call the functions and print the results
```

Expected result

Stack before popping: ['Vanna', 'Devi', 'Sok']. Returned names: ['Sok', 'Devi', 'Vanna']. The original stack is now empty.

Check and explain

Test marks of 74, 75, and 76, an empty dictionary, and cases where nobody or everybody qualifies. Why is 75 an important test? What happens when a dictionary key is reused?

Python refresher Classes and objects

Read these two refresher pages before Task 3. Run the examples in a separate practice file. They are complete examples, so leave their code unchanged on the first run.

Class and object

A class describes a kind of object: the data it stores and the actions it can perform. An object is one instance created from that class. Each object can store its own values.

```
class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return "Hello, " + self.name

first = Student("Vanna")
second = Student("Tena")
print(first.name)      # Vanna
print(second.greet())  # Hello, Tena
```

Read the example

- class Student defines the class. Student("Vanna") creates an object and Python calls `__init__` to set its starting values.
- self refers to the object receiving the method call. In `__init__`, `self.name = name` stores the supplied name on that object.
- An attribute is a named value on an object. `first.name` reads the name attribute using a dot. `first` and `second` have separate name attributes.
- A method is a function defined in a class. Calling `second.greet()` passes `second` as `self` automatically; you do not supply `self` yourself.

Try it

Set `first.name = "Mina"`, then print both names. Predict which name changes. Call `first.greet()` and explain which object `self` refers to during that call.

Python refresher References and None

A variable can refer to an object. Assigning another variable to it does not make a new copy. Both names can refer to the same object.

A node stores data and a link

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

first = Node("A")
second = Node("B")
first.next = second
head = first

print(head.value)           # A
print(head.next.value)     # B
print(head.next.next is None) # True
```

Here, next is an attribute holding a reference to another node. first.next = second links the nodes; it does not copy second. head refers to the first node. The last node has next = None, meaning there is no following node.

Follow the links

```
current = head
while current is not None:
    print(current.value)
    current = current.next
```

The loop prints A, then B. current moves along the links until it becomes None. Use is None or is not None to check for a missing node. Do not read current.value after current becomes None.

Try it

Set head.value = "Changed" and print first.value. Explain the result. Then draw first, second, and their next links. In Task 3, each node stores an employee dictionary instead of a letter.

Task 3 Employee singly linked list

File: task3_employees.py

Store employee records using linked nodes. Each node contains an employee dictionary and a next reference. Employee numbers must be unique.

Part A Follow two nodes

Use the Node class on the next page. Create employee 101 (Vanna, salary 50000, department 1) and employee 102 (Tena, salary 60000, department 2). Link 101 to 102 and assign the first node to head.

Predict head.employee['emp_no'], head.next.employee['emp_no'], and head.next.next. Draw the nodes and their links before running your code.

Part B Implement the methods

1. add_employee(emp_no, name, salary, dept_no): append a new node. Return True on success or False for a duplicate employee number.
2. search_employee(emp_no): return the matching dictionary or None if absent.
3. remove_employee(emp_no): unlink the matching node. Return True if removed or False if absent.
4. all_employees(): traverse the nodes and return their dictionaries in order. Return [] when empty. Print records in the calling code.

Hint

Start traversal at self.head and move with current.next. Removing the first node changes self.head. For another node, connect its previous node to its next node.

Use linked nodes for storage. A temporary Python list returned by all_employees() is allowed. Assume valid field types for this task.

Task 3 Starter code and checks

```
class Node:
    def __init__(self, employee):
        self.employee = employee
        self.next = None

class EmployeeList:
    def __init__(self):
        self.head = None

    def add_employee(self, emp_no, name, salary, dept_no):
        # Keys: emp_no, name, salary, dept_no
        # TODO: reject duplicates, then append a Node
        raise NotImplementedError

    def search_employee(self, emp_no):
        # TODO: traverse and return a dictionary or None
        raise NotImplementedError

    def remove_employee(self, emp_no):
        # TODO: update links and return True or False
        raise NotImplementedError

    def all_employees(self):
        # TODO: collect records by following next links
        raise NotImplementedError
```

Expected result and checks

Add 101, 102, and 103: display that order. Search 102: return its complete record.
Add 102 again: return False without changing the list.

With a fresh list for each case, remove the first, middle, and last employee.
Remove the only node: head becomes None. Search or remove 999: return None or False respectively.

Check an empty list too. Explain which link changes when removing a middle node and how your search knows when it has reached the end of the list.

Challenge 1 Book stack

Optional file: challenge1_books.py

Represent each book as a dictionary with book_no, book_name, and price. Reuse the stack behavior from Task 1.

1. Push a complete book record. Pop returns the top record or None if empty.
2. Display books from top to bottom without removing them.
3. After testing, add a repeating menu: push, pop, display, exit.

Starter code

```
def push(stack, book):
    # TODO: add the complete book record
    raise NotImplementedError

def pop(stack):
    # TODO: return the top book or None
    raise NotImplementedError

def display(stack):
    # TODO: print top first without changing stack
    raise NotImplementedError

stack = []
# TODO: test with fixed records, then add a menu
```

Input rules

Require an integer book number, a non-empty title, and a finite non-negative price. Duplicate book numbers are allowed. Invalid input must preserve the stack and allow another attempt.

Expected result and checks

Push book A, then book B. Display shows B before A; pop returns B. Display twice and check that both books remain before popping. Also test empty pop and invalid menu choices or book details.

Challenge 2 Hospital priority queue

Optional file: challenge2_hospital.py

Build a simplified queue simulation using heapq. Each patient has an ID, name, severity, DOB, and gender. A severity of 1 is most urgent and 5 is least urgent.

Queue rules

- Serve lower severity numbers first. For equal severity, serve patients in arrival order.
- IDs are non-empty strings and unique among patients currently waiting. Reject a repeated ID even if the DOB differs.
- Different IDs may share a DOB. A treated patient may be added again for a new visit.
- Store DOB as YYYY-MM-DD text. Use M, F, or Other for gender. Calendar validation is an extra extension.

Functions to implement

1. add_patient: return True when added or False for a duplicate waiting ID.
2. treat_patient: remove and return the next record, or None if empty.
3. show_next_patient: return the next record without removal, or None if empty.
4. After fixed-data checks pass, add a repeating menu to add, treat, peek, or exit.

Hint

Store heap entries as (severity, arrival_number, patient_record). Use a unique increasing number for every accepted arrival. Track waiting IDs in a set and update it when a patient is treated.

Reject invalid severity, empty names or IDs, and unsupported gender values before changing the heap or set.

Challenge 2 Starter code and checks

```
import heapq

class EmergencyRoom:
    def __init__(self):
        self.patients = []
        self.waiting_ids = set()
        self.arrival_number = 0

    def add_patient(self, name, severity, dob,
                    id_number, gender):
        # TODO: validate, check ID, and push a heap entry
        raise NotImplementedError

    def treat_patient(self):
        # TODO: pop the heap and update waiting_ids
        raise NotImplementedError

    def show_next_patient(self):
        # TODO: peek without removal; handle empty heap
        raise NotImplementedError

# TODO: fixed-data checks first, then the menu
```

Expected result

Add Zara (severity 2), Ben (severity 1), then Amy (severity 1), using distinct IDs and valid remaining fields. Treatment order must be Ben, Amy, Zara.

Check and explain

Peek twice: both calls show Ben and three patients remain. Test an empty queue, duplicate IDs with different DOBs, shared DOBs with different IDs, and adding a treated ID again.

Try 0, 6, and hello as severity input in the menu. Explain why using the name as a tie-breaker would serve Amy before Ben.